

C# pentru cine știe C++

Doar ce e diferit · exact cât îți trebuie pentru temă · Andrei

Fișa asta nu te învață C# de la zero — știi deja să programezi. Îți spune doar **unde C# se poartă altfel decât C++**, la lucrurile pe care le folosești chiar acum. Citește-o o dată întreagă, apoi ține-o lângă tine când faci tema.

Ce e identic — nu trebuie să reînveți nimic: `if / else if / else`, `for`, `while`, `switch` · operatori `+` `-` `*` `/` `%` · `==` `!=` `<` `<=` `>` `>=` · `&&`, `||`, `!` · `++`, `--`, `+=` · acoladele și punctul-și-virgulă · comentariile `//` și `/* */` · **împărțirea a două numere întregi taie zecimalele**, exact ca în C++.

1. Structura fișierului

C++	C#
<code>.h + .cpp, #include</code>	un singur <code>.cs</code> , fără header
<code>#include <iostream></code>	<code>using System;</code>
<code>include guard #pragma once</code>	nu există, nu trebuie
declari înainte de a folosi	ordinea nu contează deloc
<code>namespace X { }</code>	la fel, dar e obligatoriu în practică

`using System;` **nu e** `#include`. Nu copiază niciun fișier în al tău — spune doar „când scriu `Console`, mă refer la `System.Console`”. E o scurtătură de nume, nimic mai mult. De asta poți șterge un `using` nefolosit fără nicio consecință.

2. Tipuri

C++	C#	Observație
<code>int</code>	<code>int</code>	în C# e garantat 32 de biți, mereu
<code>double, bool, char</code>	la fel	—
<code>std::string</code>	<code>string</code>	fără <code>std::</code> , cu s mic
<code>nullptr</code>	<code>null</code>	—
<code>int*, int&</code>	nu se folosesc	vezi secțiunea 5

Capcană: `==` pe șiruri.

În C++, `==` pe două `char*` compară adrese, nu conținut — de asta folosești `strcmp` sau `std::string`. În C#, `==` pe `string` compară **întotdeauna conținutul**. Deci la exercițiul 7 din temă scrii liniștit:

```
if (txtUser.Text == "andrei" && txtParola.Text == "1234")
```

3. Intrare și ieșire — cea mai mare schimbare

În C++ programul **cere** datele când vrea el: ajunge la `cin >> x` și se oprește până tastezi. Într-o aplicație cu ferestre nu există așa ceva. Programul stă și nu face nimic; datele sunt **deja pe ecran**, în casete, iar codul tău pornește doar când utilizatorul apasă un buton.

C++	C# Windows Forms
<code>cin >> nr;</code>	<code>txtNr.Text</code> — citești când vrei tu
<code>cout << rezultat;</code>	<code>lblRezultat.Text = ...;</code> sau <code>MessageBox.Show(...)</code>
<code>int main()</code> execută de sus în jos	fiecare buton are metoda lui, care rulează la click

Nu mai ai un fir unic de execuție de la prima la ultima linie. Ai **o grămadă de bucăți mici de cod** care se declanșează independent, în orice ordine alege utilizatorul. De asta contează atât de mult starea: la a doua apăsare a unui buton, lumea nu mai e cum era la prima.

4. Conversia text → număr

În C++, `cin >> nr` face conversia singur. În C#, `TextBox.Text` e **întotdeauna** `string` și conversia e treaba ta.

C++	C#	Ce face la input greșit
<code>std::stoi(s)</code>	<code>int.Parse(s)</code>	aruncă excepție → închide aplicația
— nu are echivalent —	<code>int.TryParse(s, out int x)</code>	întoarce false, tu decizi

```
// gresit - inchide aplicatia daca caseta e goala
int nr = Int32.Parse(txtNr.Text);

// corect
if (int.TryParse(txtNr.Text, out int nr))
{
    // aici nr sigur are o valoare buna
}
else
{
    MessageBox.Show("Introdu un numar.");
}
```

`out` nu există în C++. E ca și cum ai trimite un parametru prin referință (`int&`) ca să primești un rezultat pe lângă cel returnat: metoda întoarce `bool` („am reușit?”) și îți umple variabila din `out`. Declarația chiar acolo, `out int nr`, e doar scurtătură — variabila există și după `if`.

5. Clase, obiecte și memorie

Capcană: fără ; după acoladă de clasă.

În C++ scrii `class X { ... };`. În C# punctul-și-virgula de la final **nu se pune**. E prima greșeală pe care o face oricine vine din C++.

C++	C#
<code>public:</code> ca secțiune	<code>public</code> pe fiecare membru, separat
<code>class Copil : public Parinte</code>	<code>class Copil : Parinte</code>
funcții libere, în afara claselor	nu există — totul stă într-o clasă
<code>new + delete</code>	<code>new</code> singur; nu există <code>delete</code>
destructor <code>~X()</code>	de regulă nu-l scrii

Nu există `delete`. Când un obiect nu mai e folosit de nimeni, îl curăță singur garbage collectorul, la un moment pe care nu-l alegi tu. Nu ai scurgeri de memorie din uitare, dar nici control asupra momentului eliberării.

Nu scrii niciodată `*` sau `->`. O variabilă de tip clasă e deja o referință — se comportă ca un pointer, dar accesul e mereu cu punct:

```
// C++
Form2* f = new Form2();
f->Show();

// C#
Form2 f = new Form2();
f.Show();
```

Consecința, care contează la temă: `int`, `double`, `bool` se **copiază** la atribuire; obiectele (formele, de exemplu) **nu** — două variabile pot arăta spre aceeași fereastră.

Proprietăți — nu există în C++

În C++ ai fi scris `getNum()` și `setNum()`. C# are ceva între câmp și metodă: se declară ca metodă, se folosește ca variabilă.

```
public string Raspuns
{
    get { return txtRaspuns.Text; }
}
```

Din afară scrii `f.Raspuns`, fără paranteze. Tot ce ai folosit până acum — `btn.Text`, `lbl.ForeColor`, `form.StartPosition` — sunt exact asta: în spatele fiecărei atribuirii rulează cod, nu e o simplă scriere în memorie.

Constructori

La fel ca în C++: metodă cu numele clasei, fără tip de retur. O singură capcană:

Capcană: constructorul fără parametri dispare.

Ca în C++ — în clipa în care scrii `public SalutForm(string nume)`, cel gol nu mai e generat automat. Iar Designer-ul din Visual Studio **are nevoie** de el ca să deschidă forma în modul vizual. Deci scrie-le pe amândouă:

```
public SalutForm()
{
    InitializeComponent();
}

public SalutForm(string nume)
{
    InitializeComponent();
    lblSalut.Text = "Buna, " + nume + "!";
}
```

6. Șiruri de caractere

C++	C#
<code>s1 + s2</code>	la fel
<code>std::to_string(nr)</code>	<code>nr.ToString()</code> — orice are <code>.ToString()</code>
<code>s.length()</code>	<code>s.Length</code> — proprietate, fără paranteze
— nu are —	<code>\$"Suma este {suma}"</code> — interpolare

Interpolarea îți scurtează mult codul. În loc de `nr1 + "+" + nr2 + "=" + suma` scrii:

```
MessageBox.Show($"{nr1}+{nr2}={suma}");
```

Semnul `$` dinaintea ghilimelelor e obligatoriu — fără el, acoladele rămân simple caractere.

Capcană: Text vrea string.

`lblRezultat.Text = suma;` **nu compilează** dacă `suma` e `int`. C# nu convertește singur numărul în text, așa cum nici C++ nu-ți lasă `std::string s = 5;`. Scrie `suma.ToString()`.

7. Cele patru capcane de sintaxă, pe scurt

- `;` după acolada de clasă — **nu se pune**.
- `Main`, `Length`, `Show` — metodele și proprietățile încep cu **literă mare**. C# e case-sensitive ca și C++, deci `main` nu e `Main`.
- `public` se scrie pe fiecare membru, nu ca secțiune cu `:`.
- Nu există funcții în afara claselor. Dacă vrei ceva ce în C++ ar fi fost o funcție liberă, o pui ca metodă în clasa formei.

Ce NU-ți trebuie încă (le vei întâlni, dar nu la tema asta): `List<T>` în loc de `std::vector`, `foreach`, interfețe, `try / catch`, LINQ. Le luăm când ajungem la ele.